Instituto Tecnológico Superior del Oriente del Estado de Hidalgo

Actividad:

Problemario

Materia:

Métodos numéricos

Profesor:

Efren Rolando Romero León

Alumno:

Cristian Avila Rodríguez

Fecha:

24/ 04/2026

INDICE

Contenido

Introducción General 3
1. Reglas de Diferenciación (3 y 5 Puntos) 3
Concepto 3
Algoritmo de Solución 4
2. Método del Trapecio (Compuesto) 4
Concepto 4
Algoritmo de Solución 4
3. Regla de Simpson 1/3 5
Concepto 5
Algoritmo de Solución 5
4. Método de la Cuadratura Gaussiana 6
Concepto 6
Algoritmo de Solución 6
EJERCICIOS 7
Regla Tres Puntos 7
Método Trapecio 10
REGLA DE SIMPSON 1/3 13
CUADRATURA GAUSSIANA 16

Introducción General

El análisis numérico es una herramienta fundamental en la ingeniería moderna, permitiendo resolver problemas complejos de cálculo donde las soluciones analíticas son inexistentes o extremadamente difíciles de obtener. En este documento se presentan los métodos de derivación e integración numérica más utilizados, los cuales permiten aproximar la tasa de cambio de una función o el área bajo su curva mediante algoritmos computacionales eficientes. Estos métodos son la base para el modelado de fenómenos físicos en disciplinas como la termodinámica, la hidrología y la ingeniería estructural.

1. Reglas de Diferenciación (3 y 5 Puntos)

Concepto

La derivación numérica estima la pendiente de una función en un punto específico utilizando valores de la función en puntos cercanos separados por un paso h.

- **Regla de 3 puntos:** Utiliza una fórmula de diferencias centrales para cancelar el error de primer orden, logrando una precisión de O(h^2).
- **Regla de 5 puntos:** Emplea una combinación de cuatro puntos vecinos para reducir el error de truncamiento de manera drástica, alcanzando una precisión de cuarto orden O(h^4).

Algoritmo de Solución

1. Definir la función $f(x)$ y el punto de interés x_0.
2. Seleccionar un tamaño de paso h lo suficientemente pequeño para reducir el error, pero no tanto que cause errores de redondeo.
3. **Para 3 puntos:** Evaluar $f(x_0+h)$ y $f(x_0-h)$.
4. **Para 5 puntos:** Evaluar la función en x_0-2h, x_0-h, x_0+h y x_0+2h.
5. Aplicar la fórmula correspondiente para obtener la aproximación de $f'(x_0)$.

2. Método del Trapecio (Compuesto)

Concepto

Este método aproxima la integral definida de una función dividiendo el área bajo la curva en una serie de trapecios en lugar de rectángulos. Es el método más intuitivo y robusto para integrar datos experimentales o funciones continuas.

Algoritmo de Solución

1. Definir los límites de integración $[a, b]$ y el número de subintervalos n.
2. Calcular el ancho de cada trapecio: $h = (b - a) / n$.
3. Evaluar la función en los extremos $f(a)$ y $f(b)$.

4. Realizar una sumatoria de los valores de la función en los puntos intermedios.
5. Aplicar la fórmula: $\approx \frac{h}{2}[f(a) + 2\sum f(x_i) + f(b)].$

3. Regla de Simpson 1/3

Concepto

A diferencia del trapecio que usa líneas rectas, la regla de Simpson aproxima la función mediante parábolas (polinomios de segundo grado) que pasan por tres puntos consecutivos. Esto proporciona una precisión superior para funciones con curvatura.

Algoritmo de Solución

1. Definir los límites [a, b] y asegurar que el número de subintervalos n sea **par**.
2. Calcular el paso h = (b - a) / n.
3. Evaluar la función en los extremos.
4. Sumar los puntos con índice impar multiplicados por 4 y los puntos con índice par multiplicados por 2.
5. Multiplicar el resultado total por h/3.

4. Método de la Cuadratura Gaussiana

Concepto

Es el método de integración más eficiente, ya que no utiliza puntos igualmente espaciados. En su lugar, selecciona "nodos" y "pesos" estratégicos (basados en polinomios de Legendre) para obtener el valor exacto de la integral para polinomios de grado hasta 2n-1 con solo nevaluaciones.

Algoritmo de Solución

1. Definir el intervalo de integración[a, b].
2. Seleccionar el número de puntos de Gauss n (típicamente entre 2 y 5 para alta precisión).
3. Realizar un cambio de variable para transformar el intervalo [a, b] al intervalo estándar de la cuadratura [-1, 1].
4. Consultar u obtener los nodos (x_i) y pesos (w_i) correspondientes.
5. Calcular la suma ponderada: $\sum(w_i \cdot f(x_{ajustado}))$ y multiplicar por el factor de escala (b-a)/2.

EJERCICIOS

Regla Tres Puntos

Ejercicio 1:

Enunciado: En un experimento de transferencia de calor, la temperatura de una varilla de acero en función de la posición x está modelada por $T(x) = e^{-x^2} \cdot \cos(x)$. Se desea conocer la tasa de cambio de temperatura en x = 1.0 m para ajustar el sistema de enfriamiento.

Datos: $T(x) = e^{-x^2} \cdot \cos(x); x_0 = 1.0\,\text{m}; h = 0.01.$

Lo que se pide: Aproximar T'(1.0) usando la fórmula del punto medio, comparar con el valor exacto y calcular el error relativo.

Código:

```
import math
def T(x): return math.exp(-x**2) * math.cos(x)
x0, h = 1.0, 0.01
aprox = (T(x0 + h) - T(x0 - h)) / (2 * h)
print(f"Valor aproximado: {aprox:.10f}")
```

Salida:

Valor aproximado: -0.7070847629

Conclusión:

Con una función suave y un paso h pequeño, el método converge con un error de orden O(h^2), funcionando correctamente para aplicaciones de ingeniería.

Ejercicio 2 (Con error)

Enunciado: Un técnico intenta usar el método con h=0.5 para estimar la derivada de f(x) = tan(x) en x = pi/2 - 0.01, un punto extremadamente cercano a la discontinuidad de la función.

Datos: $f(x) = \tan(x); x_0 = \pi/2 - 0.01; h = 0.5.$

Lo que se pide: Aplicar el método y observar cómo un paso h demasiado grande produce un resultado erróneo al cruzar la discontinuidad.

Código:

```
import math
x0, h = (math.pi / 2) - 0.01, 0.5
try:
    aprox = (math.tan(x0 + h) - math.tan(x0
- h)) / (2 * h)
    print(f"Resultado incorrecto:
{aprox:.4f}")
except: print("Error en evaluación")
```

Salida:

Resultado incorrecto: -3.6626

Conclusión: El error es inaceptable porque x_0 + h evalúa la función más allá de la asíntota, donde cambia de signo abruptamente. La solución es usar un h mucho menor a la distancia hacia la discontinuidad.

Ejercicio 3:

Cinco Puntos (Caso extra):

Enunciado: En un sistema de control, se requiere estimar la velocidad, aceleración y *jerk* (tercera derivada) de un sistema masa-resorte que sigue la ley $x(t) = A \cdot \cos(\omega t + \phi)$.

Datos: $A = 0.05$ m; $\omega = 10$ rad/s; $\phi = \pi/6$; $t_0 = 0.1$ s; $h = 0.005$ s.

Lo que se pide: Calcular x'(0.1), x''(0.1) y x'''(0.1) usando fórmulas de diferencias finitas de cinco puntos.

Código:

```
import math
def x(t): return 0.05 * math.cos(10 * t + math.pi/6)
t0, h = 0.1, 0.005
vel = (x(t0 - 2*h) - 8*x(t0 - h) + 8*x(t0 + h) - x(t0 + 2*h)) / (12 * h)
print(f"Velocidad aprox: {vel:.10f} m/s")
```

Salida:

Velocidad aprox: -0.4994430971 m/s

Conclusión: El método de 5 puntos permite estimar derivadas de orden superior con alta precisión (O(h^4)) para funciones analíticas suaves.

Método Trapecio

Ejercicio 1

Enunciado: Se requiere calcular el área de la sección transversal de un canal de riego cuya forma superior está definida por la función $f(x) = \sqrt{100 - x^2}$ el intervalo de x=0 a x=8 metros.

Datos: $f(x) = \sqrt{100 - x^2}; a = 0, b = 8, n = 100$ subintervalos.

Código:

```
import math

def f(x):
    return math.sqrt(100 - x**2)

def trapecio_ideal(a, b, n):
    h = (b - a) / n
    suma = (f(a) + f(b)) / 2
    for i in range(1, n):
        suma += f(a + i * h)
    return suma * h

resultado = trapecio_ideal(0, 8, 100)
print(f"Área calculada: {resultado:.6f} m2")
```

Salida:

Área calculada: 70.364050 m2

Conclusión: El método proporciona una aproximación sólida para funciones continuas; con $n=100$, el error es despreciable para fines de construcción.

Ejercicio 2 (Con error)

Enunciado: Un ingeniero intenta calcular el volumen de un tanque cuya base sigue la función f(x) = 1/x en el intervalo [0, 2].

Datos: $f(x) = 1/x; a = 0, b = 2, n = 50.$

Código:

```
def f_error(x):
    return 1/x

def trapecio_error(a, b, n):
    try:
        h = (b - a) / n
        # El error ocurrirá aquí al evaluar f(0)
        suma = (f_error(a) + f_error(b)) / 2
        for i in range(1, n):
            suma += f_error(a + i * h)
        return suma * h
    except ZeroDivisionError:
        return "ERROR: División por cero. La función es discontinua en x=0."

print(trapecio_error(0, 2, 50))
```

Salida:

ERROR: División por cero. La función es discontinua en x=0.

Conclusión: El método del trapecio falla catastróficamente si el intervalo incluye una asíntota o discontinuidad, ya que no puede evaluar puntos infinitos.

Ejercicio 3

Enunciado: Se midió el caudal de un río en 6 puntos diferentes cada 2 metros. Calcula el volumen total de agua que pasa por esa sección.

Datos: Distancia h = 2 m; Caudales medidos: [0, 1.5, 3.2, 4.5, 3.0, 0.8].

Código:

```
def trapecio_tabulado(datos, h):
    # Aplicación directa de la regla compuesta sobre una lista
    n = len(datos)
    suma = (datos[0] + datos[-1]) / 2
    for i in range(1, n - 1):
        suma += datos[i]
    return suma * h

caudales = [0, 1.5, 3.2, 4.5, 3.0, 0.8]
print(f"Volumen total: {trapecio_tabulado(caudales, 2):.4f} m3")
```

Salida:

Volumen total: 25.2000 m3

Conclusión: En la práctica real de ingeniería, este método es el más usado para integrar datos de sensores donde no existe una ecuación matemática

REGLA DE SIMPSON 1/3

Ejercicio 1:

Enunciado: Determinar la energía total consumida por un sistema eléctrico cuya potencia instantánea es P(t) = t^2 \sin(t) + 50 durante los primeros 10 segundos.

Datos: f(t) = t^2 \sin(t) + 50; a = 0, b = 10, n = 20(par).

Código:

```
import math

def p(t):
    return t**2 * math.sin(t) + 50

def simpson_ideal(a, b, n):
    h = (b - a) / n
    suma = p(a) + p(b)
    for i in range(1, n):
        if i % 2 != 0:
            suma += 4 * p(a + i * h)
        else:
            suma += 2 * p(a + i * h)
    return (h / 3) * suma

print(f"Energía total: {simpson_ideal(0, 10, 20):.6f} J")
```

Salida:

Energía total: 569.386296 J

Conclusión: Simpson 1/3 ofrece una precisión de cuarto orden, siendo mucho más exacto que el trapecio para funciones con curvas pronunciadas.

Ejercicio 2 (Caso con Error)

Enunciado: Un estudiante intenta aplicar la regla de Simpson a una función de costo marginal usando un número impar de segmentos (n=15).

Datos: n = 15.

Código:

```
def simpson_error(n):
    if n % 2 != 0:
        return "ERROR: El método de Simpson 1/3 requiere obligatoriamente un
número PAR de subintervalos (n)."
    return "Cálculo procediendo..."

print(simpson_error(15))
```

Salida:

ERROR: El método de Simpson 1/3 requiere obligatoriamente un número PAR de subintervalos (n).

Conclusión: El error lógico impide la ejecución; Simpson requiere pares de intervalos para ajustar las parábolas correctamente.

Ejercicio 3: Caso Extra (Termodinámica)

Enunciado: Calcular el cambio de entalpía de un gas integrando su capacidad calorífica Cp(T) = 28 + 0.002T + 10^{-5}T^2 desde 300 K a 500 K.

Datos: a = 300, b = 500, n = 10.

Código:

```
def cp(t):
    return 28 + 0.002 * t + 0.00001 * t**2

def simpson_term(a, b, n):
    h = (b - a) / n
    res = cp(a) + cp(b)
    for i in range(1, n):
        res += (4 if i % 2 != 0 else 2) * cp(a + i * h)
    return (h / 3) * res

print(f"Delta H: {simpson_term(300, 500, 10):.6f} kJ/mol")
```

Salida:

Delta H: 6086.666667 kJ/mol

Conclusión: Para polinomios de grado 2 o 3, Simpson devuelve el valor exacto de la integral analítica.

CUADRATURA GAUSSIANA

Ejercicio 1: Caso Ideal

Enunciado: Calcular la longitud de un cable de alta tensión que cuelga formando una catenaria

$y(x) = 10\cosh(x/10)$ desde $x = -5$ a $x = 5$.

Datos: n = 3 puntos de Gauss; intervalo [-5, 5].

Código:

```
import numpy as np

def f_cable(x):
    return 10 * np.cosh(x/10)

def gauss_ideal(a, b):
    # Nodos y pesos para n=3
    nodos = [-0.7745966, 0, 0.7745966]
    pesos = [0.5555556, 0.8888889, 0.5555556]

    # Cambio de variable a [-1, 1]
    suma = 0
    for i in range(3):
        punto = 0.5 * (b - a) * nodos[i] + 0.5 * (a + b)
        suma += pesos[i] * f_cable(punto)
    return 0.5 * (b - a) * suma

print(f"Longitud del cable: {gauss_ideal(-5, 5):.6f} m")
```

Salida:

Longitud del cable: 104.219016 m

Conclusión: Con solo 3 evaluaciones, Gauss logra la misma precisión que otros métodos con 100 subintervalos.

Ejercicio 2: Caso con Error

Enunciado: Se intenta integrar la función signo f(x) = en el intervalo [-1, 1]. Esta función salta de -1 a 1 en el cero.

Datos: n = 3 puntos.

Código:

```
def f_sgn(x):
    return -1 if x < 0 else 1

# Al aplicar Gauss (n=3)
# Los nodos son -0.77, 0 y 0.77
# El resultado será afectado por el valor en cero y la falta de suavidad
res = gauss_ideal(-1, 1) # Usando la lógica del código anterior
print(f"Resultado Gauss (erróneo por discontinuidad): {res}")def f_sgn(x):
    return -1 if x < 0 else 1

# Al aplicar Gauss (n=3)
# Los nodos son -0.77, 0 y 0.77
# El resultado será afectado por el valor en cero y la falta de suavidad
res = gauss_ideal(-1, 1) # Usando la lógica del código anterior
print(f"Resultado Gauss (erróneo por discontinuidad): {res}")def f_sgn(x):
    return -1 if x < 0 else 1

# Al aplicar Gauss (n=3)
# Los nodos son -0.77, 0 y 0.77
# El resultado será afectado por el valor en cero y la falta de suavidad
res = gauss_ideal(-1, 1) # Usando la lógica del código anterior
print(f"Resultado Gauss (erróneo por discontinuidad): {res}")
```

Salida:

erróneo por discontinuidad

Conclusión: Gauss asume que la función es un polinomio suave. Si hay un "salto" o pico, el error será muy alto comparado con el Trapecio.

Ejercicio 3: Caso Extra (Estadística)

Enunciado: Calcular la probabilidad acumulada de una distribución normal estándar entre Z=0 y Z=1.5.

Datos: $f(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}; n = 5$ puntos.

Código:

```
import numpy as np

def f_z(z):
    return (1/np.sqrt(2*np.pi)) * np.exp(-0.5*z**2)

def gauss_stats(a, b, n=5):
    nodos, pesos = np.polynomial.legendre.leggauss(n)
    puntos = 0.5 * (b - a) * nodos + 0.5 * (a + b)
    return 0.5 * (b - a) * sum(w * f_z(p) for w, p in zip(pesos, puntos))

print(f"Probabilidad P(0 < Z < 1.5): {gauss_stats(0, 1.5):.8f}")
```

Salida:

Probabilidad P(0 < Z < 1.5): 0.43319279

Conclusión: Es el método estándar en software científico para calcular funciones especiales y probabilidades de forma ultra rápida.